

Programming and Algorithms

Aula 12: Sequências

Estrutura de repetição definida: for

Prof. Ettore Baldini-Neto
ettore.neto@faculdadeimpacta.com.br

Aulas passadas

- Estrutura de repetição indefinida: While
- Variáveis contadoras e acumuladoras
- Controle de laços com flags booleanas
- Laços infinitos
- Simulação de repeat until com while True
- Laços aninhados

Programa da Aula

- O que são sequências?
  - Strings, listas, tuplas e intervalos
  - Sequências mutáveis e imutáveis
  - Sequências homogênas e heterogênas
  - Percorrendo sequências por meio de seus índices
- Loop for: estrutura de repetição definida
- Percorrendo sequências por meio de seus itens com o loop for.

Motivações

- Na medida em que a complexidade dos códigos aumenta torna-se evidente a necessidade de manipular e melhorar o gerenciamento de dados.
- Procuramos por estruturas mais eficientes para manipulação de dados. Sob este ponto de vista a estrutura de repetição definida **for** é especialmente projetada para percorrer listas.
- Ela é parecida com a estrutura de repetição indefinida while, **mas a cada repetição utiliza um elemento diferente de uma dada lista, conforme veremos.**

Introdução

- Quando o problema exige a manipulação de uma quantidade grande de dados, deixa de fazer sentido armazená-los em variáveis simples.
- Para isso vamos aprender o conceito de sequências que é um recurso importante e bastante usado em Python.
- Um ponto importante é que: não basta armazenarmos uma grande quantidade de dados se não pudermos acessá-los de maneira simples e rápida. Veremos que isso pode ser feito com a estrutura **for.**
- Nesta estrutura, o número de repetições do bloco de código é definida **antes do loop ser iniciado**, ao contrário da estrutura while.

Sequências

- **Sequências** são um tipo de variável que permite o armazenamento de vários valores que são acessados por um **índice.** O próprio nome sequência indica uma ordem na estrutura.
- Em outras, palavras, podemos imaginá-las como uma variável com diversos compartimentos que são chamados **itens da sequência.**
- Cada item é identificado por dois valores:
- **o nome da variável à qual a sequência foi atribuída**
- **um número inteiro referente à sua posição na sequência, índice.**
- Em Python, **os índices são contados a partir do zero**, **em ordem crescente.** Logo, o primeiro item da sequência tem índice 0, o segundo tem índice 1 e assim por diante.

Sequências

Particularidade do Python: existência de índices negativos ou regressivos. Ou seja, os itens são associados a dois índices. Um que marca a posição relativa ao início da sequência (positivo) e o outro que marca a posição relativa ao fim da sequência (negativo).

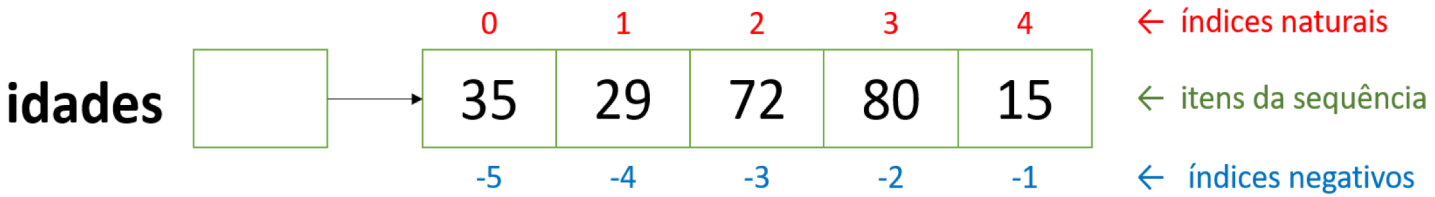


Figura 9.1: Uma sequência atribuída a uma variável. Fonte: Elaborado pelo autor.

Acessando um ítem de uma sequência

Para acessar um item específico da sequência basta indexá-la com a posição em que está o item desejado.

```
>>> idades = [10, 12, 13, 14, 18, 25]
>>> idades
[10, 12, 13, 14, 18, 25]
>>> idades[0]
10
>>> idades[4]
18
>>> idades[-1]
25
>>> idades[-2]
18
>>> idades[-5]
12
```

Sequências: A função len( )

Obs. O Python possui diversas funções e métodos que nos ajudam trabalhar com sequências, como por exemplo, a função len( ) **que recebe como argumento uma sequência e devolve o número natural que indica o tamanho (”comprimento”) dessa sequência.**

```
>>> idades = [10, 14, 12, 23, 45, 65]
>>> len(idades)
6
```

Sequências mutáveis e imutáveis

Sequências mutáveis: cada item se comporta como uma variável comum, ou seja, pode receber atribuições, sobrescrevendo seus valores iniciais.

```
>>> L = [3.1, 7.8, 1.2, 10.0, 9.5]
>>> L[0]
3.1
>>> L[1]=6.0
>>> L
[3.1, 6.0, 1.2, 10.0, 9.5]
```

- Note que aqui temos uma sequência mutável, neste caso uma lista.
- Sobrescrevemos o valor do índice 1 que é de 7.8 para 6.0.

Sequências mutáveis e imutáveis

Sequências imutáveis: Uma vez definida a sequência, seus itens **não podem** ser modificados

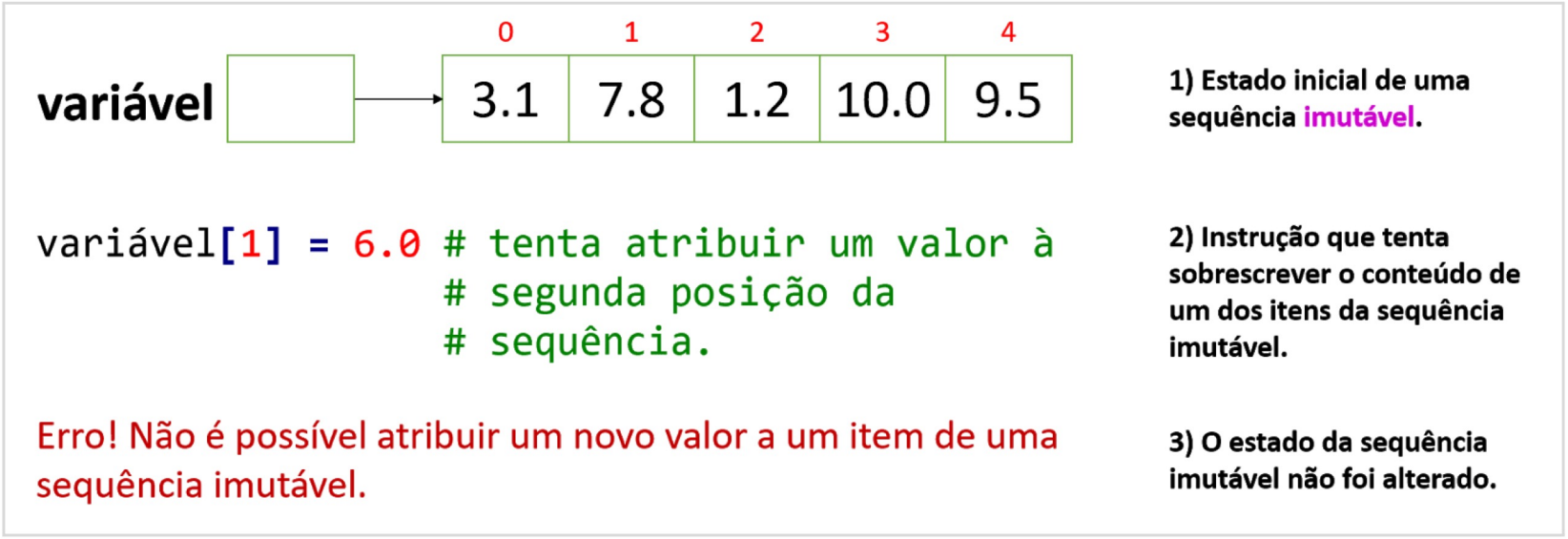


Figura 9.3: Tentativa de modificação de item de uma sequência imutável.
Fonte: Elaborado pelo autor.

Sequências mutáveis e imutáveis

Apesar de os itens das sequências imutáveis **não poderem ser modificados**, a variável que faz referência à sequência **pode** receber um novo valor. Neste caso, sobrescreve-se o valor da variável e não o valor de um item da sequência.

```
>>> variavel = (1,2,3,4,5)
>>> variavel
(1, 2, 3, 4, 5)
>>> variavel[0] = 2
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    variavel[0] = 2
TypeError: 'tuple' object does not support item assignment
>>> variavel = 6
>>> variavel
6
```

Note que a sequência original foi apagada e outro valor atribuído à variável.

Sequências homogêneas e heterogêneas

Em sequências **homogêneas**, todos os itens devem ser do **mesmo** tipo

Em sequências **heterogêneas**, os itens podem ser de tipos **distintos.**

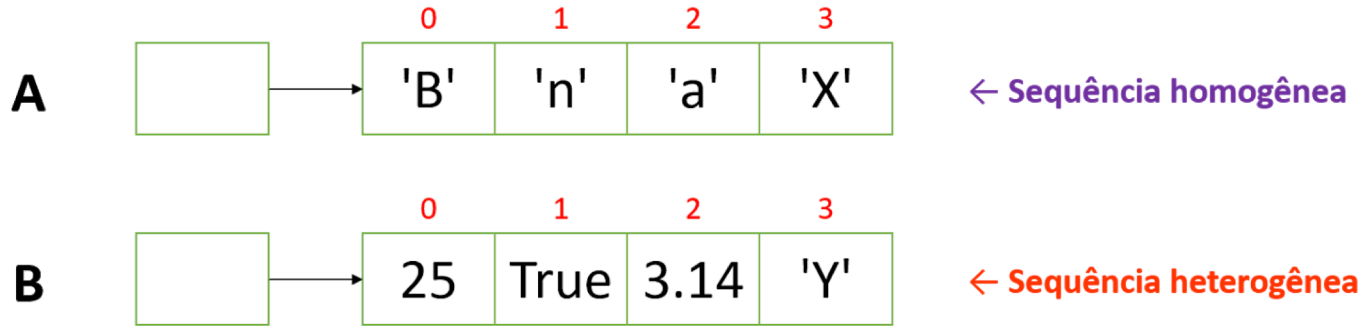


Figura 9.5: Exemplos de sequências homogênea e heterogênea.
Fonte: Elaborado pelo autor.

Tipos de sequências em Python

São três tipos básicos de sequências**: listas (list), tuplas (tuples) e intervalos(range).**

Há também um tipo específico para amazenar strings e outros tipos de sequências que servem para manipulação de dados.

Vamos falar nesta aula dos tipos:

- string
- lista
- tupla
- intervalo.

Tipos de Sequências em Python

Strings: São sequências **imutáveis e homogêneas**. Assim como em outras sequências é possível acessar um item específico da string, indexando-a. Podem ser definidas de várias formas:

- entre pares de apóstrofos
- entre pares de aspas
- entre pares de três apóstrofos ou aspas, permitindo que tenham múltiplas linhas
- Com o construtor tipo **str**( ) que permite a conversão de outros tipos para string.

Tipos de Sequências em Python

```
>>> texto = ''#atribui uma string vazia à variável
>>> texto
''
>>> texto = 'O céu é azul!'
>>> texto
'O céu é azul!'
>>> len(texto)
13
>>> texto[0]
'O'
>>> texto[0]='x'
Traceback (most recent call last):
  File "<pyshell#6>", line 1, in <module>
    texto[0]='x'
TypeError: 'str' object does not support item assignment
>>>
```

Listas

Sequências de listas são **mutáveis** e **potencialmente heterogêneas**

```
>>>
>>> lista = [ ]
>>> lista
[]
>>> lista = [3.14, True, 20, 'oi']
>>> lista
[3.14, True, 20, 'oi']
>>> len(lista)
4
>>> lista[2]
20
>>> lista[-2]
20
>>> lista[3] = 'tchau'
>>> lista
[3.14, True, 20, 'tchau']
>>>
```

Tuplas (tuples)

- Sequências do tipo **tuples** são **imutáveis e potencialmente heterogêneas.**
- Seu uso é comum quando queremos garantir que a **sequência não seja modificada**, como em chaves de dicionários, ou ao ser passada como argumento para funções que produzem efeitos colaterais (veremos nas próximas aulas).

Tuplas

```
>>>
>>> tuple = ( )
>>> tuple
()
>>> tuple = (123,)
>>> tuple
(123,)
>>> tuple = (3.14, True, 29, 'oi!')
>>> tuple
(3.14, True, 29, 'oi!')
>>> len(tuple)
4
>>> tuple[1]
True
>>> tuple[1] = False
Traceback (most recent call last):
  File "<pyshell#27>", line 1, in <module>
    tuple[1] = False
TypeError: 'tuple' object does not support item assignment
>>>
```

Intervalo (range)

Sequências do tipo range são **imutáveis e homogêneas** gerando um intervalo de números inteiros. Intervalos são criados com o construtor **range( ),** acompanhado dos parâmetros **início, fim e passo,** que devem ser **números inteiros.**

Existem 3 variações para o uso de range:

- range(início, fim, passo) com 3 parâmetros, gerando a sequência no intervalo [início..fim[, variando de passo em passo.
- range(início, fim) com 2 parâmetros gerando uma sequência de passo 1 no intervalo [início, fim[, variando de 1 unidade.
- range(fim) com um parâmetro, gerando um sequência crescente com itens no intervalo [0, fim[, variando de 1 unidade.

Intervalo: Atenção

- Note que o intervalo **é aberto à direita**, o que significa que o valor fim **não pertence** à sequência
- Um intervalo gerado pode ser vazio como, por exemplo, range(10, 10)
- O passo pode ser negativo, gerando uma sequência **decrescente**, desde que **início > fim**, caso contrário será gerado um intervalo vazio.
- **Note que passo 0 é inválido**



Intervalo

```
>>> intervalo = range(0)
>>> intervalo
range(0, 0)
>>> intervalo = range(5,10,2)
>>> intervalo
range(5, 10, 2)
>>> lista = list(intervalo)
>>> lista
[5, 7, 9]
>>> tupla = tuple(intervalo)
>>> tupla
(5, 7, 9)
>>> intervalo[-1]
9
>>> intervalo[0] = 77
Traceback (most recent call last):
  File "<pyshell#9>", line 1, in <module>
    intervalo[0] = 77
TypeError: 'range' object does not support item assignment
>>>
```

Percorrendo sequências

- Percorrer uma sequência significa acessar sistematicamente diversos de seus itens seguindo alguma ordem.
- Essa ação costuma ser feita com a utilização de loops (laços de repetição). Vamos tentar entender esses conceitos através do próximo exemplo.

Exemplo 1: Crie um programa que receba de entrada quatro salários, exiba a média salarial e os salários que estão abaixo dessa média. **Não vamos usar sequências ainda.**

Percorrendo sequências: força bruta

```
soma = 0
salario_0 = float(input('Salario: R$ '))
soma += salario_0
salario_1 = float(input('Salario: R$ '))
soma += salario_1
salario_2 = float(input('Salario: R$ '))
soma += salario_2
salario_3 = float(input('Salario: R$ '))
soma += salario_3

media = soma / 4
print(f'A média salarial é R$ {media:.2f}')

if salario_0 < media:
    print(f'Abaixo da média: R$ {salario_0:.2f}')
if salario_1 < media:
    print(f'Abaixo da média: R$ {salario_1:.2f}')
if salario_2 < media:
    print(f'Abaixo da média: R$ {salario_2:.2f}')
if salario_3 < media:
    print(f'Abaixo da média: R$ {salario_3:.2f}')
```

Percorrendo sequências: utilizando lista

```
salarios = [0,0,0,0]
soma = 0

salarios[0] = float(input('Salario: R$ '))
soma += salarios[0]
salarios[1] = float(input('Salario: R$ '))
soma += salarios[1]
salarios[2] = float(input('Salario: R$ '))
soma += salarios[2]
salarios[3] = float(input('Salario: R$ '))
soma += salarios[3]

media = soma / 4
print(f'A média salarial é R$ {media:.2f}')

if salarios[0] < media:
    print(f'Abaixo da média R$ {salarios[0]:.2f}')
if salarios[1] < media:
    print(f'Abaixo da média R$ {salarios[1]:.2f}')
if salarios[2] < media:
    print(f'Abaixo da média R$ {salarios[2]:.2f}')
if salarios[3] < media:
    print(f'Abaixo da média R$ {salarios[3]:.2f}')
```

Percorrendo sequências

Podemos percorrer listas **variando apenas seu índice**, uma vez que o nome da variável que faz referência à sequência é o mesmo. Podemos então usar o nome de uma variável como índice e alterá-lo de acordo com a necessidade de indexação.

Percorrendo sequências: com while

```
salarios = [0,0,0,0]
soma = 0

i = 0   # variável que representa (será usada como) o índice
while i < 4:
    salarios[i] = float(input('Salário: R$ '))
    soma += salarios[i]
    i += 1

media = soma / 4
print(f'Média = R$ {media:.2f}')

i = 0  # variável que será usada como índice
while i < 4:
    if salarios[i] < media:
        print(f'Abaixo da média: R$ {salarios[i]:.2f}')
    i += 1
```

Estrutura de repetição for

Este loop é utilizado **quando sabemos de antemão a quantidade de vezes que o bloco de código deve ser repetido, ou para percorrer sequências.**

Sintaxe em python: (Para cada ítem na sequência faça....)

```
for < variável > in < sequência>:
    < bloco de código >
```

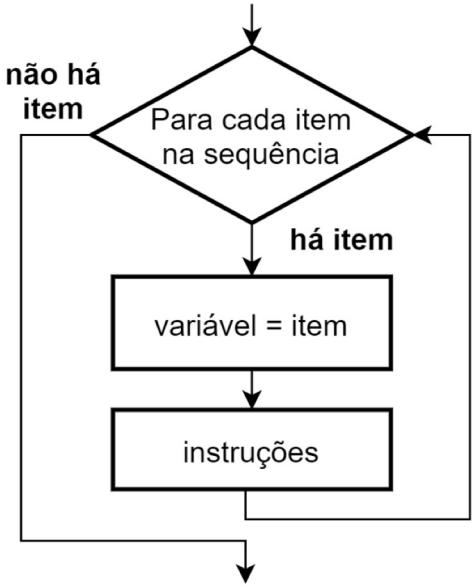
a indentação é a mesma que já utilizamos em outras estruturas.

Estrutura de repetição for

- Note que < sequência > é uma sequência de qualquer tipo e tamanho, podendo ser até vazia.
- Por sua vez, < variável > é uma **variável de controle** que, caso não exista previamente, será criada no início da execução do loop se a sequência não for vazia.
- Note que o **primeiro item será atribuído a essa variável dando início à primeira execução do < bloco de código >.**
- Caso exista uma variável com um identificador igual à < variável > seu valor será sobrescrito pela atribuição do primeiro item de < sequência >.

Estrutura de repetição for

O laço for executa seu bloco de código uma vez para cada item da sequência atribuído à < variável >. Não é recomendável alterar < sequência > durante a execução do for.



Para cada item da sequência associada ao laço **`for`**, o item é atribuído à variável de controle.

Após a atribuição, as instruções do bloco de código são executadas e o fluxo da execução retorna para o início do laço.

Quando não houver mais itens inéditos na sequência para serem atribuídos, o laço é encerrado.

Figura 9.10: Representação em fluxograma da estrutura de repetição `for`.
Fonte: Elaborado pelo autor.

Estrutura de repetição for

Vamos reescrever a codificação anterior usando o laço for.

Vamos usar um **método** muito requisitado quando trabalhamos com listas: o método **append( ), que anexa ao final da própria lista um novo item.**

Assim, é desnecessário que a lista inicie já com todas as posições preenchidas fazendo com que o Python cuide da alocação de memória de maneira mais eficiente.

Visualize a execução no python tutor.

```python
salarios = [ ]
soma = 0

for _ in range(4):
    salario = float(input('Salário: R$ '))
    soma += salario
    salarios.append(salario)

media = soma / 4
print(f'Média = R$ {media:.2f}')

for salario in salarios:
    if salario < media:
        print(f'Abaixo da média: R$ {salario:.2f}')
```

Estrutura de repetição for

Note que no primeiro laço, o sublinhado (_) foi utilizado como identificador da variável de controle. Isso indica que **não há interesse nos valores assumidos por essa variável.** Poderíamos ter usado qualquer outro nome válido, mas este é o padrão quando o objetivo do loop **é apenas ser executado um número fixo de vezes sem pretensão de utilizar os valores da sequência atribuídos à variável de controle.**

Note que range retorna um intervalo [0, 4) e portanto o laço será executado 4 vezes e, como usamos o append para incluir os itens na lista, **não precisamos nos preocupar em gerenciar os índices.**

Estrutura de repetição for

Exemplo 1: Crie um programa que exiba o alfabeto em letras minúsculas e maiúsculas em uma linha cada.

Solução:

```
for letra in 'abcdefghijklmnopqrstuvwxyz':
    print(letra,end = '' )

print( )

for letra in 'ABCDEFGHIJKLMNOPQRSTUVWXYZ':
    print(letra, end = '')
```

Estrutura de repetição for

Exemplo 2: Crie um programa que leia 5 nomes e exiba a quantidade de nomes que começam com vogal

Solução:

```
nomes = [ ]
for _ in range(5):
    nomes.append(input('Nome: '))

qtd = 0
for nome in nomes:
    if (nome[0] == 'A' or nome[0] == 'E' or nome[0] == 'I' or
        nome[0] == 'O' or nome[0] == 'U'):
        qtd += 1

print(f'{qtd} dos nomes começam com vogal ')
```

Estrutura de repetição for

Exemplo 3 : Crie um programa que exiba os dias da semana

Solução:

```
dias = ('domingo', 'segunda', 'terça', 'quarta', 'quinta',
        'sexta', 'sábado')
for dia in dias:
    print(dia)
```

Estrutura de repetição for

Exemplo 4: Crie um programa que exiba, em ordem crescente, os pares de 10 até 100 e, em ordem decrescente, os ímpares de 100 até 10.

Solução:

```
for par in range(10,101,2):
    print(par)

print()

for impar in range(99, 10, -2):
    print(impar)
```

Estrutura de repetição for

Exemplo 4: Crie um programa que exiba, em ordem crescente, os pares de 10 até 100 e, em ordem decrescente, os ímpares de 100 até 10.

Solução:

```
for par in range(10,101,2):
    print(par)

print()

for impar in range(99, 10, -2):
    print(impar)
```

Estrutura de repetição for

Exercício 1 * : Crie um programa que entra com 5 notas (entre 0 e 10) de um aluno usando listas e calcule sua média aritmética. Caso esta média seja menor que 5, atribua o conceito F a ela, caso esteja no intervalo [5,6) atribua conceito E, no intervalo [6,7) conceito D, no intervalo [7, 8) conceito C, no intervalo [8,9) conceito B e o conceito A, caso seja maior ou igual a 9. Imprima a média com uma casa decimal.

Estrutura de repetição for

Exercício 2 : Modifique o código abaixo para o programa perguntar quantos salários serão inseridos. A seguir, ele receberá cada salário, calculará a média e exibirá todos aqueles que estão abaixo dela.

```
salarios = [ ]
soma = 0

for _ in range(4):
    salario = float(input('Salário: R$ '))
    soma += salario
    salarios.append(salario)

media = soma / 4
print(f'Média = R$ {media:.2f}')

for salario in salarios:
    if salario < media:
        print(f'Abaixo da média: R$ {salario:.2f}')
```

Estrutura de repetição for

Exercício 3 : Modifique a mesma codificação anterior para o programa receba salários indefinidamente até que o usuário digite -1 como salario. A seguir ele receberá cada salário, calculará a média e exibirá todos aqueles que estão abaixo dela.

FIM